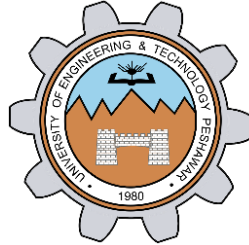


Lab # 09

Binary Search Trees



Spring 2026

Data Structures And Algorithms Lab

Submitted by: **M Mohsin Khan**

Registration No.: **24PWCSE2382**

Class Section: **C**

Submitted to:

Engr. Abdullah Hamid

June 21 , 2026

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Task : Binary Search Trees:

```
#include <iostream>
using namespace std;
// Define a BST Node
struct Node
{
    int data;
    Node *left;
    Node *right;
};
// a) Creation: Create a new node
Node *createNode(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->left = nullptr;
    newNode->right = nullptr;
    return newNode;
}
// b) Insertion
Node *insert(Node *root, int value)
{
    if (root == nullptr)
        return createNode(value);
    if (value < root->data)
        root->left = insert(root->left,
value);
    else if (value > root->data)
        root->right = insert(root->right,
value);
    return root;
}
// c) Find Minimum
Node *findMin(Node *root)
{
    while (root != nullptr && root->left !=
nullptr)
        root = root->left;
    return root;
}
// d) Find Maximum
Node *findMax(Node *root)
{
    while (root != nullptr && root->right
!= nullptr)
        root = root->right;
    return root;
}
```

```
// e) Delete Element
Node *deleteNode(Node *root, int key)
{
    if (root == nullptr)
        return root;
    if (key < root->data)
        root->left = deleteNode(root->left,
key);
    else if (key > root->data)
        root->right = deleteNode(root->
right, key);
    else
    {
        // Node with one or no child
        if (root->left == nullptr)
        {
            Node *temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == nullptr)
        {
            Node *temp = root->left;
            delete root;
            return temp;
        }
        // Node with two children
        Node *temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->
right, temp->data);
    }
    return root;
}
// f) Post-order Traversal
void postOrder(Node *root)
{
    if (root == nullptr)
        return;
    postOrder(root->left);
    postOrder(root->right);
    cout << root->data << " ";
}
// g) Pre-order Traversal
void preOrder(Node *root)
{
    if (root == nullptr)
```

```

        return;
    cout << root->data << " ";
    preOrder(root->left);
    preOrder(root->right);
}
// h) In-order Traversal
void inOrder(Node *root)
{
    if (root == nullptr)
        return;
    inOrder(root->left);
    cout << root->data << " ";
    inOrder(root->right);
}

int main()
{
    Node *root = nullptr;

    // Insert elements
    int elements[] = {50, 30, 20, 40, 70,
60, 80};

    for (int i = 0; i < 7; i++)
    {
        root = insert(root, elements[i]);
    }

    cout << "Binary Search Tree Created
Successfully\n\n";

    cout << "In-Order Traversal (Sorted
Order): ";
    inOrder(root);

    cout << "\nPre-Order Traversal: ";
    preOrder(root);

    cout << "\nPost-Order Traversal: ";
    postOrder(root);

    cout << "\n\nMinimum Element: "
<< findMin(root)->data;

```

```

    cout << "\nMaximum Element: "
<< findMax(root)->data;

    // Delete leaf node
    cout << "\n\nDeleting Leaf Node
(20)...\n";
    root = deleteNode(root, 20);

    cout << "In-Order Traversal: ";
    inOrder(root);

    // Delete node with one child
    cout << "\n\nDeleting Node With One
Child (30)...\n";
    root = deleteNode(root, 30);

    cout << "In-Order Traversal: ";
    inOrder(root);

    // Delete node with two children
    cout << "\n\nDeleting Node With Two
Children (50)...\n";
    root = deleteNode(root, 50);

    cout << "In-Order Traversal: ";
    inOrder(root);

    cout << "\n\nFinal BST Traversals\n";

    cout << "In-Order: ";
    inOrder(root);

    cout << "\nPre-Order: ";
    preOrder(root);

    cout << "\nPost-Order: ";
    postOrder(root);

    cout << endl;

    return 0;
}

```

Terminal output:

```
PS D:\4th Semester\DSA\lab_09_BST> .\a.exe
Binary Search Tree Created Successfully

In-Order Traversal (Sorted Order): 20 30 40 50 60 70 80
Pre-Order Traversal: 50 30 20 40 70 60 80
Post-Order Traversal: 20 40 30 60 80 70 50

Minimum Element: 20
Maximum Element: 80

Deleting Leaf Node (20)...
In-Order Traversal: 30 40 50 60 70 80

Deleting Node With One Child (30)...
In-Order Traversal: 40 50 60 70 80

Deleting Node With Two Children (50)...
In-Order Traversal: 40 60 70 80

Final BST Traversals
In-Order: 40 60 70 80
Pre-Order: 60 40 70 80
Post-Order: 40 80 70 60
```

RUBRICS (LABS/ PROJECTS):

Criteria & Point Assigned	Outstanding 4	Acceptable 3	Considerable 2	Below Expectations 1	Score
Valuing related to work, including responsibility, and professionalism.	Demonstrates excellent responsibility and professionalism in task management. Shows strong commitment to quality work, meeting the deadline, and comprehensive, well-organized effective documentation following all guidelines with exemplary attention to detail.	Demonstrates some responsibility and professionalism. Displays moderate commitment to quality work, meets the deadline and shows sufficient thoroughness in documentation, adhering to guidelines, with some room for enhancement.	Demonstrates limited effort in responsibility and professionalism. Misses the deadline with notable inconsistency in quality work and shows insufficient thoroughness or clarity in documentation, necessitating improvements.	Demonstrates a lack of responsibility and professionalism. Misses the deadline and lacks quality work. Documentation is incomplete, disorganized, or unclear.	
Ability to apply techniques, methods, and procedures effectively.	Executes lab tasks with exceptional skill to design/develop solution and shows proficiency in lab techniques and procedures.	Executes lab tasks with adequate skill to design/develop solution and shows proficiency in lab techniques and procedures.	Executes lab tasks with minimal skill to design/develop solution and shows moderate proficiency in lab techniques and procedures.	Executes lab tasks with limited skill to design/develop solution and lacks proficiency in lab techniques and procedures.	

Demonstrate understanding and relevant Knowledge.	Demonstrates exceptional understanding of concepts with clear explanation of investigation and analysis of the objectives and outcomes.	Demonstrates a good understanding of concepts with some explanations of investigation and analysis that align with the lab's objectives and outcomes.	Demonstrates a basic understanding of key concepts but lacks clarity or depth in explaining the investigation and analysis, with some inconsistencies.	Demonstrates limited understanding of concepts with unclear explanations of investigation and analysis that do not align with the lab's objectives and outcomes.	
---	--	--	---	---	--